CITL Factbook + Transcribe + Translation

Engineering Technical Manual | Build snapshot: **2026-03-11** | Audience: Engineering, IT Security, Academic Technology, Compliance Reviewers

Manual Index

Positioning: This system is designed to keep instructional data processing on institutional infrastructure by default.

Legal note: This manual is technical guidance and does not replace legal counsel review.

1) Scope and Operating Model

This application provides a local-first assistant for three primary workloads:

- **Reference Q&A:** deterministic and retrieval-augmented answers over local corpus data.
- **Audio capture and transcription:** in-app microphone capture and local speech-to-text.
- **Translation:** offline machine translation with optional study-pair generation.

Primary design goals

- Reduce cloud exposure of classroom artifacts.
- Keep outputs grounded in local corpus evidence where possible.
- Support reproducible model governance (Modelfile-based customization).
- Offer a practical operations surface for instructors and technical staff.

Non-goals

- No claim of full legal compliance automation.
- No guarantee of correctness without corpus coverage and quality checks.
- No replacement for institutional records-retention policy and access control policy.

2) Architecture and Data Flows

High-level runtime flow

```
GUI (Tkinter)
  -> Factbook tab asks question
    -> factbook_assistant_gui._query_factbook(...)
```

```
    -> citl_factbook_query.answer_question(...)
      1) deterministic local truth path (entity-locked where available)
      2) embedding retrieval from local index
      3) local Ollama generation with grounding constraints
      4) safe fallback if generation unavailable

Audio tab
  -> start_recording/stop_recording (sounddevice)
  -> faster-whisper transcription (local CPU)

Translate tab
  -> argostranslate translate(...)
  -> optional language-pack install

Library/Models tab
  -> corpus indexing scripts
  -> Modelfile authoring
  -> ollama pull/create/list lifecycle
```

Reference QA reliability layers

- **Generic reference pipeline:** citl_factbook/reference_ingest.py, reference_db.py, reference_query.py.

- **Entity-locked answering:** entity detection + canonical-field routing + constrained lookup.
- **Fallback strategy:** deterministic answer path preferred; safe refusal when reliability checks fail.

Core reliability principle: Do not emit unconstrained high-rank chunk text as final answer when grounding cannot be verified.

3) Important Files and Responsibilities

Paths are relative to repository root.

File	Primary Responsibility	Operational Notes
factbook_assistant_gui.py	Root launcher wrapper that delegates to package GUI.	Ensures factbook-assistant/ is on sys.path.
factbook-assistant/ factbook_assistant_gui.py	Main desktop UI (tabs, buttons, model management, query dispatch).	Central control-plane for most operator workflows.
factbook-assistant/ citl_factbook_query.py	Reference-answer orchestration, embedding retrieval, Ollama generation, fallback behavior.	Contains strict grounding safeguards and model resolution logic.
factbook-assistant/citl_factbook/ reference_ingest.py	Generic corpus parser/ingester into entity + section	Supports configurable regex/header/canoni

File	Primary Responsibility	Operational Notes
	structures.	cal-field mappings.
factbook-assistant/citl_factbook/reference_db.py	SQLite schema + FTS operations for corpora/entities/sections.	Contains structured lookup and entity-scoped FTS fallback.
factbook-assistant/citl_factbook/reference_query.py	Query planning (entity, intent, field), execution, rendering.	Implements comparison mode and reliability outcomes.
factbook-assistant/citl_factbook/reference_config.py	Default parser/field config and optional JSON/YAML override load.	Key extension point for non-Factbook reference corpora.
factbook-assistant/citl_factbook/cli.py	CLI entrypoint for offline reference QA.	Uses reference pipeline first, legacy retrieval fallback second.
factbook-assistant/citl_factbook/retrieve.py	Legacy keyword scoring retriever over JSONL index.	Basic fallback, not the preferred reliability path.
factbook-assistant/citl_factbook/ollama_client.py	Simple Ollama generate client using env-configured model/host.	Legacy/simple generation utility.
factbook-assistant/	Builds local	Auto-defaults --src/--

File	Primary Responsibility	Operational Notes
build_corpus_index.py	embedding index from file/dir sources.	out for GUI-triggered calls.
factbook-assistant/ build_factbook_index.py	Factbook-specific embedding index build path.	Triggered from Library/Models tab button.
factbook-assistant/ citl_audio_ffmpeg_graceful_v2.py	Audio device discovery, recording start/stop, diagnostics.	Primary recording path uses sounddevice; FFmpeg used for diagnostics/compatibility.
factbook-assistant/citl_translation.py	Offline translation and language-pack install wrappers.	Built on argostranslate.
factbook-assistant/ citl_transcribe_lecture.py	Lecture CLI (record, transcribe, optional summarize).	Useful for scripted/terminal workflows.
factbook-assistant/ citl_class_capture.py	Class capture UI with recording + whisper transcription + VTT export.	Alternate capture experience.
factbook-assistant/citl_modelfile.py	Parses Modelfile metadata and extracts base/system fields.	Backs GUI modelfile load/apply behavior.

File	Primary Responsibility	Operational Notes
factbook-assistant/citl_theme.py	Theme palette definitions and apply helper.	Includes accessible dark/light palette set.
factbook-assistant/citl_app_sync.py	Cross-platform repository sync utility for external/USB targets.	Excludes large caches/models/audio by default.
data/citl/config_citl-mainframe-All-Series.json	Local persisted operational profile (model, host, theme, modelfile path).	Primary provenance and runtime default source on this host.
data/citl/modelfiles/Modelfile.citl-custom4qw	CITL model customization profile (FROM base + SYSTEM prompt).	Documents local custom-model behavior and constraints.
factbook-assistant/tests/test_factbook_reliable_qa.py	Reliability tests for factbook workflow (goldens + negative controls).	Includes Iran-vs-UAE contamination regression guard.
factbook-assistant/tests/test_reference_pipeline_generic.py	Generic non-Factbook reference-pipeline behavior tests.	Validates entity-locking and unknown-field fail-safe behavior.

4) GUI Control and Button Reference (Main Window)

Toolbar controls

Control	Handler	Technical Action	State Impact
Load Modelfile	on_load_modelfile()	Opens file picker, parses metadata/system/base, applies profile to UI, refreshes model lists, may trigger auto-graft check.	Updates config keys: model/base/theme/bot/language/system prompt.
USB Sync	on_open_usb_sync()	Launches app sync utility window for cross-repo synchronization.	No model state change; may copy files to external target.
Theme dropdown	on_theme_changed()	Applies palette from citl_theme.	Writes theme to config.

Factbook tab

Control	Handler	Technical Action	Dependencies
Refresh Models	on_refresh_models()	Fetches /api/tags from Ollama, merges JSON model list + saved models, de-duplicates, repopulates combos.	Ollama reachable or JSON model list fallback.
Load Model JSON	on_load_model_json()	Loads external JSON and extracts model	Readable JSON file.

Control	Handler	Technical Action	Dependencies
		names recursively from common keys.	
Ask Factbook	on_ask_factbook()	Captures question/model/host; async call to _query_factbook() and writes response to output pane.	citl_factbook_query, local index files, Ollama (optional depending on deterministic path).
Translate Answer	on_send_factbook_to_translate()	Copies Factbook output text into Translate tab source and switches tab.	None.

Audio / Transcribe tab

Control	Handler	Technical Action	Output
Refresh	refresh_devices()	Enumerates input devices; updates saved device selection.	Device list + status message.
Reset Saved	reset_saved_device()	Clears persisted audio device from config and re-	Config update.

Control	Handler	Technical Action	Output
		enumerates.	
Diagnostics	on_diagnostics()	Prints audio diagnostics (including dshow note on Linux).	Diagnostics text pane.
Start Recording	on_start()	Calls start_recording(), creates WAV in recordings directory, caches handle.	Live recording stream.
Stop Recording	on_stop()	Stops stream, validates WAV signal metrics, logs evidence line.	Saved WAV and metrics log.
Open Recordings Folder	on_open_recordings_folder()	Opens recordings directory via OS file manager.	Folder view.
Open Last WAV	on_open_last_wav()	Opens latest recorded WAV in system default app.	File open.
Transcribe Last WAV	on_transcribe()	Runs _transcribe_wav() using faster-whisper; writes transcript text and	Transcript in pane + .txt file.

Control	Handler	Technical Action	Output
		sidecar TXT.	
Translate Transcript	on_send_transcript_to_translate()	Copies transcript pane text into Translate tab source.	Tab switch with text seed.

Translate tab

Control	Handler	Technical Action	Notes
⇄	on_swap_languages()	Swaps source and target languages in UI vars.	No translation call.
Translate	on_translate()	Runs offline translation via citl_translation.translate() thread.	Requires installed pair.
Install Pack	on_install_lang_pack()	Calls Argos package index/update/install workflow.	May require internet once for package download.
Clear	on_clear_translate()	Clears source/result/vocabulary panes.	UI reset only.
Live translation checkbox	_on_tr_source_keyrelease()	Debounced translate call every 800ms while typing.	Optional real-time translation mode.

Library / Models tab

Control	Handler	Technical Action	Execution Detail
Add Books (txt/docx/pdf)	on_add_books()	Copies selected files into factbook-assistant/library/.	Does not auto-index; run rebuild next.
Rebuild Factbook Index	on_rebuild_factbook_index()	Launches python build_factbook_index.py in background log runner.	Writes embeddings/chunks artifacts.
Rebuild Corpus Index	on_rebuild_corpus_index()	Launches python build_corpus_index.py.	Generic corpus path.
Verify Index Health	on_verify_index()	Loads common embedding JSON files and reports detected matrix shape.	Best-effort health check.
USB App Sync	on_open_usb_sync()	Opens CITL app sync utility from this tab context.	Optional control availability.
Set As Operational	on_set_operational_from_new_name()	Promotes new model name to active operational model.	Persists to config.
Browse	on_browse_modelfile()	Selects modelfile path.	No write until save/create.

Control	Handler	Technical Action	Execution Detail
Open Folder	on_open_modelfiles_dir()	Opens Modelfile directory in file manager.	Path from config/env fallback.
Write Modelfile	on_write_modelfile()	Generates Modelfile with metadata + FROM + SYSTEM prompt.	Updates config and model selections.
Ollama: List	on_ollama_list()	Runs ollama list and streams output to log pane.	Requires ollama in PATH.
Ollama: Pull FROM	on_ollama_pull_from()	Runs ollama pull <base_model>.	Downloads base model artifact.
Ollama: Create/Update	on_ollama_create()	Ensures Modelfile exists, then runs ollama create <name> -f <modelfile>.	Verifies model appears in ollama list.

Coverage note: This section documents controls in the main application window and its four primary tabs.

5) LLM Provenance and Governance

Current local model provenance snapshot (2026-03-11)

Item	Observed Value	Evidence Path / Command
Configured	citl-custom4qw	data/citl/config_citl-mainframe-All-

Item	Observed Value	Evidence Path / Command
operational model		Series.json
Resolved runtime model	citl-custom4qw:latest	ollama list
Base model for custom model	olmo2:13b	data/citl/modelfiles/Modelfile.citl-custom4qw (FROM line)
Base model architecture	olmo2, 13.7B params, Q4_K_M	ollama show olmo2:13b
Base model system provenance text	"...built by the Allen Institute for AI"	ollama show olmo2:13b
Custom model system prompt	CITL academic support + fact-checking rules	ollama show --modelfile citl-custom4qw:latest
Embedding model	nomic-embed-text:latest	ollama list + retrieval defaults

Governance procedure for model changes

1. Update or write Modelfile via GUI (**Write Modelfile**).
2. Create model artifact via **Ollama: Create/Update**.
3. Record model hash and metadata from ollama list and ollama show.
4. Persist operational model selection in config profile.
5. Run reliability smoke tests before classroom deployment.

Why AllenAI model bases here: transparent lineage, local deployment support, open

licensing posture (Apache 2.0 in observed local model metadata), and fit for education/NGO governance requirements.

6) Privacy, SaaS Risk, and Institutional Controls

How this architecture addresses SaaS privacy concerns

Risk Area	Typical SaaS LLM Risk	CITL Local Mitigation
Student text/audio exposure	Data sent to external vendor endpoints.	Inference/transcription/translation run locally by default; no mandatory cloud API in primary workflow.
Vendor model training reuse	Potential use of prompts/logs for provider model improvement (policy-dependent).	Local-hosted models avoid automatic third-party training pipelines.
Data brokerage / repurposing	Unclear downstream sharing controls across vendor ecosystem.	Institution controls storage and transport boundaries; no inherent broker feed in local stack.
Cross-border data transfer	Opaque data residency and subprocessors.	Local deployment supports institution-controlled residency strategy.
Auditing and evidence	Limited vendor transparency into model lineage and runtime details.	Modelfile + local config + local model metadata are inspectable and reproducible.

Institutional privacy ordinance alignment (technical posture)

- Supports data minimization by keeping instructional processing local when configured accordingly.

- Supports least-privilege operations through local account and filesystem controls.
- Supports auditability through config files, model manifests, and local logs.
- Supports policy-driven retention by controlling local transcript/index storage directories.

Compliance boundary: this software can support compliance objectives (for example FERPA-style student record controls), but institutional policy, legal review, IAM controls, and endpoint security are still required.

Data handling boundaries to enforce operationally

- Do not enable outbound LLM APIs in classroom profiles unless approved by policy.
- Pin Ollama host to institution-managed local endpoint.
- Limit corpus ingestion paths to approved instructional materials.
- Apply retention schedules to data/citl/recordings and transcript outputs.
- Restrict access to config and index artifacts to authorized staff.

7) Engineer Runbook and Verification Steps

Startup validation checklist

```
1. Start Ollama service

2. Verify models:

   ollama list

3. Verify active profile:

   cat data/citl/config_citl-mainframe-All-Series.json
```

```
4. Open app and confirm model/host in Factbook tab

5. Run smoke query and verify citations + country/entity lock behavior

6. Run transcript test (30-60 sec audio)

7. Run translation test with installed language pair
```

Ollama verification chain Execute this sequence without skipping steps when validating a new profile rollout.

```
ollama list

ollama show olmo2:13b

ollama show citl-custom4qw:latest

ollama show --modelfile citl-custom4qw:latest
```

Model provenance verification commands

```
ollama list

ollama show olmo2:13b

ollama show citl-custom4qw:latest

ollama show --modelfile citl-custom4qw:latest
```

Regression checks before deployment

```
python -m unittest factbook-assistant/tests/test_factbook_reliable_qa.py

python -m unittest factbook-assistant/tests/test_reference_pipeline_generic.py
```

Release gate: if entity-locking regressions are observed (for example wrong country/entity response), block classroom release until fixed and re-tested.

8) Engineer Q&A (Tough Questions Playbook)

Q1. Is this just another chatbot wrapper?

No. It is a composite local system: deterministic reference query logic, local retrieval/indexing, local model lifecycle management, local transcription, and offline translation. The chatbot surface is only one component.

Q2. How do we prove model provenance?

Use local evidence chain: config profile (ollama_model), Modelfile FROM line, ollama list model IDs, and ollama show metadata. This establishes model identity, lineage, and local runtime artifact.

Q3. Why AllenAI/OLMo family instead of closed SaaS models?

Primary reasons are governance and controllability: inspectable model lineage, local hosting, reproducible deployment, and reduced third-party exposure of classroom artifacts.

Q4. Does the app send student data to the cloud?

In default local deployment, primary inference and processing paths are local. Administrators must still verify network policy, host configuration, and package-install phases (for example one-time language pack downloads).

Q5. Can vendor-side training on our prompts happen here?

For local-only model execution, there is no default external provider training loop. If external APIs are introduced later, this assumption changes and requires policy review.

Q6. How does this reduce data brokerage risk?

By avoiding default transmission of classroom data to ad-tech or multi-tenant SaaS ecosystems, institutional data remains in governed local infrastructure unless explicitly exported.

Q7. What institutional privacy frameworks does this support?

It supports technical controls commonly needed for student-data governance (local processing, access control, auditability, retention controls). Formal legal compliance status must be determined by institutional counsel and policy teams.

Q8. What problem does entity-locked retrieval solve?

It prevents cross-entity contamination (for example returning UAE data for an Iran question). This is critical for trust in reference-book QA workflows.

Q9. What happens when model generation endpoint is unavailable?

The system can return deterministic/local reliability-safe results or explicit inability-to-answer messages instead of hallucinated content.

Q10. Why keep the legacy retriever modules if a better pipeline exists?

Legacy modules preserve backward compatibility and can act as degraded fallback paths while migration to the generic reference pipeline continues.

Q11. Is transcription fully offline?

Speech recognition is local in current workflows. One-time dependency/model installation may require internet during setup phases.

Q12. Is translation fully offline?

After language packs are installed, translation runs offline via Argos. Initial package

retrieval is network-dependent.

Q13. What is the strongest evidence this is not locked to one book?

The reference_ingest/reference_query/reference_config modules are corpus-agnostic and tested with non-Factbook sample corpora in test_reference_pipeline_generic.py.

Q14. How do we answer claims that local models are less accurate than SaaS?

Accuracy is workload-specific. This system mitigates risk by deterministic extraction, citation requirements, entity-locking, and regression tests. For high-stakes use, domain eval sets and acceptance thresholds should drive deployment decisions.

Q15. What if instructors accidentally ingest sensitive files?

Implement ingestion policy controls: approved directories, access controls, and review workflows. The tool allows local governance; it does not enforce institution policy by itself.

Q16. What if a dean asks "why this app exists"?

It solves the gap between AI utility and institutional privacy obligations: local-first instructional assistance, auditable model governance, and practical teacher-facing workflows for Q&A, transcription, and translation.

Q17. What are the primary measurable outcomes?

Reduced external data transfer for classroom artifacts, faster instructor retrieval workflows, improved transcript availability, and repeatable provenance tracking for deployed models.

Q18. Can this meet all privacy requirements automatically?

No. It provides a strong technical substrate, but compliance requires policy, IAM, endpoint controls, network controls, retention policy, and legal governance.

Q19. How do we defend against "black box" criticism?

By providing inspectable Modelfiles, local model metadata, explicit retrieval pipelines, citation outputs, deterministic fallback logic, and regression tests tied to known failure modes.

Q20. What should trigger immediate engineering escalation?

Cross-entity answer contamination, missing-citation high-confidence answers, model identity drift, unexpected outbound traffic, or failures in reliability test suite.

9) Known Limits and Escalation Triggers

Known technical limits

- Legacy retriever path remains keyword-based and should not be treated as high-assurance by itself.
- Corpus quality and parser heuristics directly affect deterministic extraction quality.
- Audio quality, device gain, and environment noise strongly affect transcription outcomes.

Escalate immediately when observed

- Any wrong-entity answers in known deterministic question classes.
- Loss of model provenance evidence (unknown model ID/lineage in deployment).
- Operational profile drift across lab/classroom endpoints.

- Any policy exception requiring non-local API routing for student artifacts.

Document owner: CITL Engineering. Review cadence: each model or architecture change, or at least once per academic term.